

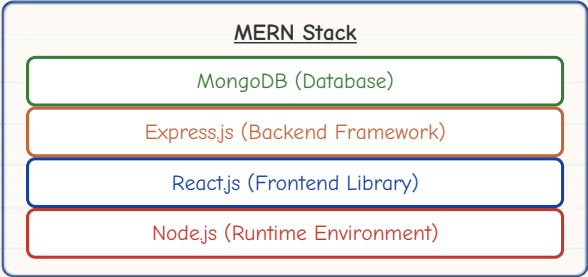
mERN Stack Notes

By Ajeet Yadav

MongoDB , Expressjs , Reactjs , Nodejs

* What is MERN Stack?

- MERN Stack is a collection of JavaScript based technologies used to build dynamic web applications.
- It allows developers to build complete full stack applications using a single language – JavaScript.
- It is open-source, fast, scalable and widely used.



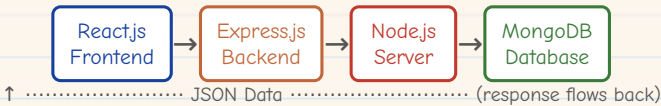
* Components of MERN Stack

- MongoDB** – NoSQL database that stores data in JSON-like documents.
- Express.js** – Web application framework for Node.js.
- React.js** – JavaScript library for building user interfaces.
- Node.js** – JavaScript runtime environment to run server-side code.

* Why MERN Stack?

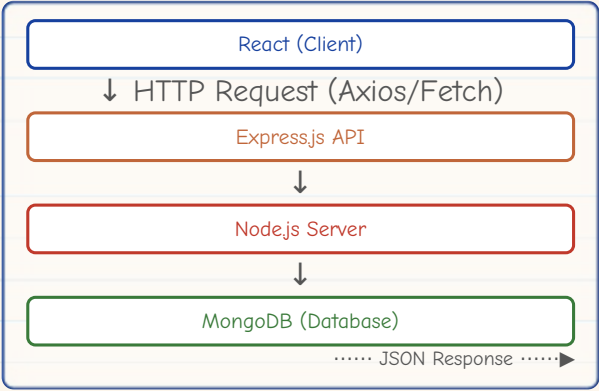
- Uses JavaScript everywhere
- High performance & speed
- Open source & large community
- Easy to learn & build
- Scalable & flexible
- Rich ecosystem & tools
- Great for Single Page Applications (SPA)
- Supports full stack development.

* MERN Architecture



* Request Flow in MERN Stack

- User interacts with Reactjs UI.
- React sends HTTP request to Expressjs API.
- Expressjs handles the request and passes it to appropriate controller.
- Controller interacts with MongoDB through Mongoose.
- Data is sent back to client as JSON.
- React updates the UI with the received data.



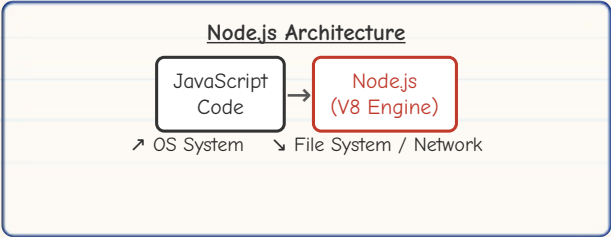
mERN Stack Notes

By Ajeet Yadav

MongoDB , Expressjs , Reactjs , Nodejs

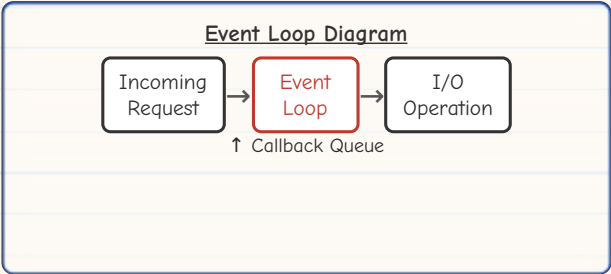
* Node.js Fundamentals

- Nodejs is a JavaScript runtime built on Chrome's V8 engine.
- It allows us to run JavaScript on the server side.
- Nodejs is non-blocking and event-driven.
- It is lightweight and highly scalable.
- It has a rich ecosystem via npm.



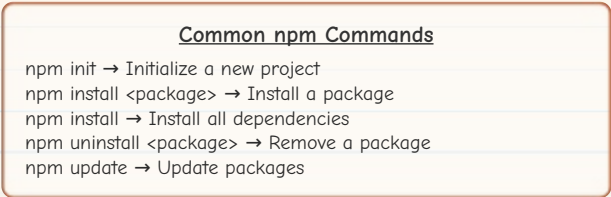
* Node.js Core Concepts

- **Event Loop** – Handles all async operations.
- **Non-blocking I/O** – Allows multiple operations to run at the same time.
- **Single Threaded** – Uses event loop to handle concurrent requests.
- **Callback Queue** – Stores callbacks to be executed.



* npm (Node Package Manager)

- npm is the default package manager for Nodejs.
- It helps in installing, updating and managing packages.
- `package.json` file contains project metadata and dependencies.



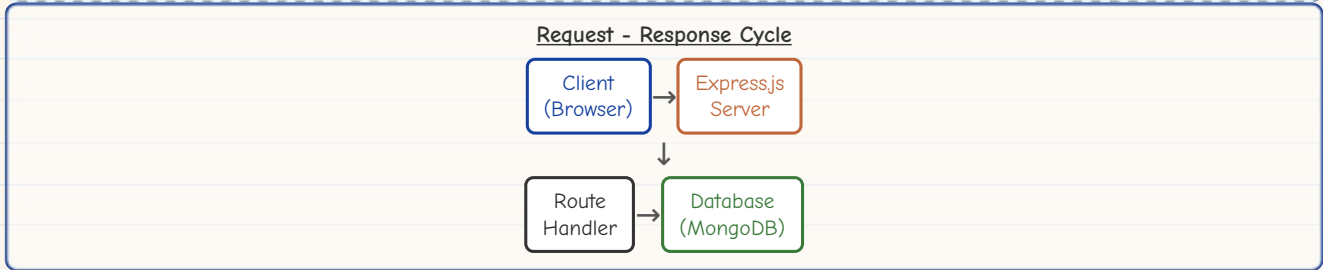
* Express.js Basics

- Expressjs is a minimal and flexible Nodejs web application framework.
- It provides a robust set of features for web and mobile applications.
- It simplifies the process of building APIs.

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello MERN!');
});

app.listen(5000, () => {
  console.log('Server running on port 5000');
});
```



★ **Key Point:** Nodejs + Expressjs helps us build fast, scalable and lightweight backend APIs for our applications.

mERN Stack Notes

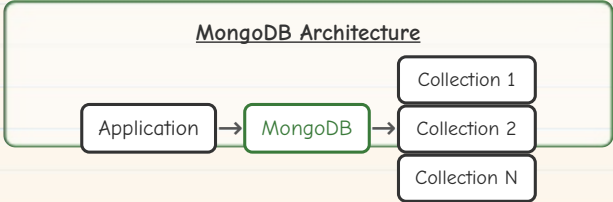
by Ajeet Yadav



MongoDB , Expressjs , Reactjs , Node.js

* MongoDB Fundamentals

- MongoDB is a NoSQL database that stores data in flexible, JSON-like documents.
- It is schema-less, scalable and high performance.
- Data is stored in Collections and Collections contain Documents



* MongoDB CRUD Operations

- Each document has a unique `_id` field.
- **Create** → Insert document
- **Read** → Find documents
- **Update** → Update documents
- **Delete** → Delete documents

```
db.users.insertOne({...}) // Create
db.users.find({...})      // Read
db.users.updateOne({...}) // Update
db.users.deleteOne({...}) // Delete
```

* Mongoose Basics

- Mongoose is an ODM (Object Data Modeling) library for MongoDB.
- It provides schema validation, easy query building and middleware support.
- It helps in structuring and interacting with MongoDB in a more efficient way.

```
// Define Schema
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: { type: Number, min: 18 },
  createdAt: { type: Date, default: Date.now }
});

// Create Model & Use
const User = mongoose.model('User', userSchema);
const user = new User({ name: 'Zaid', email: 'zaid@gmail.com' });
await user.save();
const users = await User.find();
```

* Database Relationships in MongoDB

- MongoDB is schema-less but supports relationships using embedding and referencing.
- Use **embedding** for one-to-few relationships.
- Use **referencing** for one-to-many or many-to-many relationships.

Embedding (One-to-Few)

```
{ name: "Zaid",
  posts: [
    { title: "Post 1", likes: 10 },
    { title: "Post 2", likes: 20 }
  ] }
```

Referencing (One-to-Many)

User Collection → Posts Collection

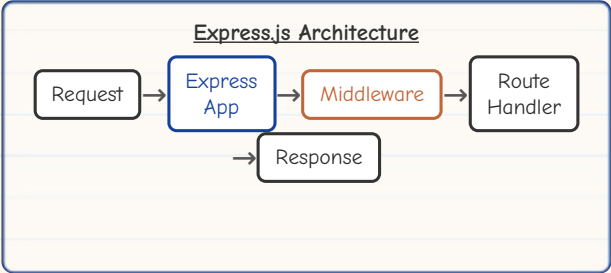
★ **Key Point:** MongoDB stores data in flexible documents, Mongoose helps in modeling and Expressjs provides APIs to interact with this data.

By Ajeet Yadav

MongoDB , Expressjs , Reactjs , Nodejs

* Expressjs – Building REST APIs

- Expressjs is a fast, unopinionated and minimalist web framework for Node.js.
- It helps in building APIs and web applications easily.
- It provides routing, middleware support, request handling and much more.
- It is the standard server framework for Node.js.



* Routing in Express.js

- Routing is used to define endpoints (URIs) and their corresponding request handlers.
- Each route can handle different HTTP methods like GET, POST, PUT, DELETE.

```
app.get('/', (req,res) => {
  res.send("Welcome to Express!");
});
app.post('/api/data', (req,res) => {
  res.json({ message: "Data received" });
});
```

Method	Description
GET	Retrieve data from the server.
POST	Send data to create a resource.
PUT	Update entire resource.
PATCH	Partially update resource.
DELETE	Delete a resource.

* Middleware in Express.js

- Middleware functions have access to the request (req), response (res) and the next middleware function.
- They can execute code, modify req/res, end the cycle, or call next().

```
app.use((req, res, next) => {
  console.log("Request:", req.method, req.url);
  next(); // pass control
});
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: "Something went wrong!" });
});
```

★ **Key Point:** Expressjs provides a powerful set of features to build RESTful APIs efficiently and handle requests and responses smoothly.

mERN Stack Notes

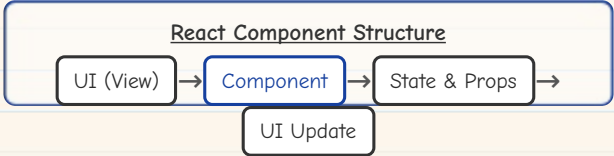
by Ajeet Yadav

mERN
Stack

MongoDB , Expressjs , React.js , Node.js

* React.js Basics

- React is a JavaScript library for building user interfaces.
- It uses components to build reusable UI.
- It follows a declarative and component-based approach.
- * It uses a Virtual DOM for better performance.



- JSX allows us to write HTML-like code inside JavaScript.
- It makes the code more readable and easy to write.
- JSX is transpiled to `React.createElement()`.

```
const element = (  
  <div className="card">  
    <h1>Hello MERN!</h1>  
    <p>Learning is fun 🐼</p>  
  </div>  
);
```

```
// Transpiled output  
const element = React.createElement(  
  "div", { className: "card" },  
  React.createElement("h1", null, "Hello MERN!")  
);
```

* React Component Types

```
// Functional Component  
function Greet() {  
  return <h1>Hello, MERN!</h1>;  
}  
export default Greet;
```

```
// Class Component  
class Greet extends React.Component {  
  render() {  
    return <h1>Hello, MERN!</h1>;  
  }  
}
```

- **Hooks** – allow functional components to use state and other React features.

* React State & Props

```
import { useState } from 'react';  
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count+1)}>  
    Count: {count}</button>;  
}
```

```
// Props are read-only, passed parent → child  
function Greet(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}  
// Usage: <Greet name="Zaid" />
```

* React Lifecycle (with Hooks)



★ **Key Point:** React makes it easy to build interactive UIs using components, state, props and reusable logic.



* State in React (vs. Data)

MERN Stack Notes

- State is data that changes over time.
- useState is a React Hook that allows us to add state to functional components.
- When state updates, the component re-renders.

```
function Counter() {
  const [count, setCount] = useState(0);
  return (<div>
    <h2>Count: {count}</h2>
    <button onClick={() => setCount(count+1)}>
      Increment</button>
    </div>);
}
```

* React Event Handling

- Events in React are written in camelCase.
- We pass a function as the event handler.
- React events are synthetic events.

```
function Button() {
  const handleClick = () => {
    alert("Button Clicked!");
  };
  return <button onClick={handleClick}>Click Me</button>;
}
```

* Conditional Rendering

```
if (isLoggedIn) {
  return <Dashboard />;
} else {
  return <Login />;
}
```

```
// Ternary
return isLoggedIn
  ? <Dashboard />
  : <Login />;
```

```
// Logical &&
return (
  isLoggedIn &&
  <Dashboard />
);
```

* Lists & Keys

```
const users = ["Zaid","Ali","Sara"];
function UserList() {
  return (<ul>
    {users.map((u,i) =>
      <li key={i}>{u}</li>)}
    </ul>);
}
```

- We use map() to render lists.
- Each item needs a unique key.
- Key helps React identify which items changed, added or removed.

* Lifting State Up

★ **Key Point:** React is all about building reusable components, managing state effectively and updating UI efficiently.

Parent
(Holds State)

mERN Stack Notes



By Ajeet Yadav

MongoDB , Express.js , React.js , Node.js

* React Forms

- In React, forms are usually handled using controlled components.
- The form data is controlled by the component's state.
- Every input field has a value and onChange handler.

```
function LoginForm() {
  const [form, setForm] = useState({email:"",
  password:""});
  const handleChange = (e) => {
    setForm({...form, [e.target.name]: e.target.value});
  };
  const handleSubmit = (e) => { e.preventDefault();
  console.log(form); };
}
```

* React Router (Client Side Routing)

- React Router is used to handle navigation in single page applications.
- It helps in defining routes and rendering components based on the URL.
- Provides <BrowserRouter>, <Routes>, <Route>, <Link> components.

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
function App() {
  return (
    <BrowserRouter>
      <Link to="/">Home</Link> | <Link to="/about">About</Link>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```



- **Browser Router** – Uses HTML5 history API.
- **Hash Router** – Uses # in URL (legacy).
- **useNavigate** – Programmatic navigation.
- **useParams** – Access dynamic route parameters.

* Connecting Frontend with Backend

- React (frontend) communicates with Express.js (backend) using HTTP APIs.
- We use fetch() or axios to make API requests. Data is usually in JSON format.

```
import axios from 'axios';
const fetchUsers = async () => {
  try {
    const res = await axios.get('http://localhost:5000/api/users');
    console.log(res.data);
  } catch (err) { console.error(err); }
};
```

```
const express = require('express');
const app = express();
app.get('/api/users', (req,res) => {
  const users = [{id:1,name:"Zaid"},{id:2,name:"Ali"}];
  res.json(users);
});
```

★ **Key Point:** React handles UI and user interaction, while Node.js + Express.js handles business logic, APIs and database operations.

mERN Stack Notes

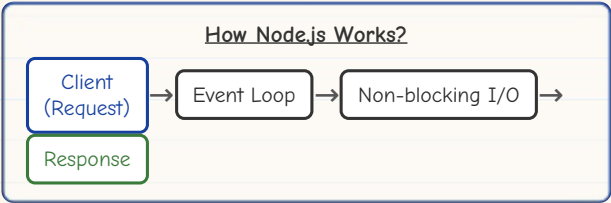


By Ajeet Yadav

MongoDB , Expressjs , Reactjs , Nodejs

* Node.js Basics

- Nodejs is a JavaScript runtime built on Chrome's V8 engine.
- It allows us to run JavaScript on the server.
- It is non-blocking, event-driven and lightweight.
- Nodejs has a rich ecosystem via npm.



* Creating a Simple Node.js Server

```
const http = require("http");
const server = http.createServer((req,res) => {
  res.writeHead(200, {'Content-Type':'text/plain'});
  res.end("Hello from Node.js Server!");
});
server.listen(5000, () => console.log("Server running on port 5000"));
```

- `createServer()` creates a server instance.
- It listens for requests on the given port.
- `res.writeHead()` sets status code and content type.
- `res.end()` sends the response to the client.

* Middleware in Expressjs

- Middleware functions have access to req, res and next middleware. They execute in the order defined. Used for logging, authentication, error handling, etc.

```
const logger = (req,res,next) => {
  console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);
  next();
};
app.use(logger);
```

- `express.json()` – Parses incoming JSON data
- `express.urlencoded()` – Parses URL-encoded data
- `express.static()` – Serves static files
- `express.Router()` – Modular routing

* Connecting MongoDB with Node.js (Mongoose)

- Steps: 1) Install mongoose 2) Connect to database 3) Define schema & model 4) Perform CRUD operations

```
const mongoose = require("mongoose");
mongoose.connect("mongodb://127.0.0.1:27017/mern_demo")
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.error(err));
const userSchema = new mongoose.Schema({name:String, email:String});
const User = mongoose.model("User", userSchema);
```

- **Create** → `Model.create(data)`
- **Read** → `Model.find()`
- **Update** → `Model.findByIdAndUpdate(id, data)`
- **Delete** → `Model.findByIdAndDelete(id)`

★ **Key Point:** Nodejs runs JS on the server, Expressjs simplifies backend development and Mongoose makes MongoDB interaction easy and productive.

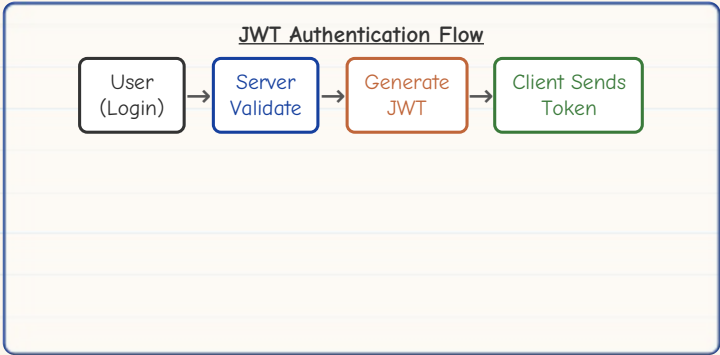
mERN Stack Notes

By Ajeet Yadav

MongoDB , Express.js , React.js , Node.js

* Authentication in MERN (JWT)

- JWT (JSON Web Token) is used for secure authentication.
- User logs in → server validates credentials → returns JWT token.
- Token is stored on client and sent with each request.
- Server verifies token and allows access to protected routes.



```
// Generate JWT
const jwt = require("jsonwebtoken");
const token = jwt.sign({id:user._id, role:user.role},
  process.env.JWT_SECRET, {expiresIn:"7d"});

// Verify JWT (Middleware)
module.exports = function(req,res,next) {
  const token = req.header("Authorization");
  if (!token) return res.status(401).json({msg:"No token"});
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch (err) { res.status(401).json({msg:"Invalid token"}); }
};
```

* Error Handling (Backend)

```
module.exports = function(err,req,res,next) {
  console.error(err.message);
  res.status(err.statusCode || 500).json({
    success: false, message: err.message || "Server Error"
  });
};
```

Code	Meaning
200	OK
400	Bad Request
401	Unauthorized
404	Not Found
500	Internal Server Error

* Environment Variables

```
# .env file
PORT=5000
MONGO_URI=mongodb://localhost:27017/mern_demo
JWT_SECRET=mySuperSecretKey123
CLIENT_URL=http://localhost:3000

require("dotenv").config();
const PORT = process.env.PORT;
const mongoURI = process.env.MONGO_URI;
```

★ **Key Point:** Authentication, error handling and environment variables make your MERN application secure, reliable and production-ready.

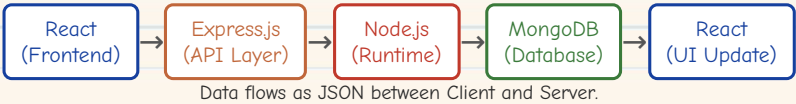
MERN Stack Notes



By Ajeet Yadav

MongoDB , Express.js , React.js , Node.js

1. MERN Architecture (End-to-End Flow)



2. React Quick Revision

- Components – Building blocks of UI.
- Props vs State – Props read-only, State mutable.
- Hooks – useState, useEffect, useContext, useReducer, useRef.
- useEffect – Runs side effects after render.
- Context API – Share data without prop drilling.
- Redux Toolkit – Global state management.

3. Backend Quick Revision

- Express Routing – app.get(), app.post()
- Middleware – Functions before route handler.
- JWT – Authentication and authorization.
- REST APIs – GET, POST, PUT, DELETE.
- MVC Pattern – Clean architecture.

4. MongoDB Quick Revision

- Collections – Groups of documents.
- Documents – JSON-like data.
- CRUD – Create, Read, Update, Delete.
- Indexes – Improve query performance.
- Aggregation – Pipeline for data processing.

5. Top MERN Interview Questions

1. What is the difference between state and props in React?
2. Explain the useEffect hook with an example.
3. How does React Router work?
4. How would you protect a route in React?
5. What is JWT and how does it work?
6. How would you handle errors in Express?
7. What are middleware and how is it used?
8. How does MongoDB store data?
9. What is a Mongoose schema?
10. How would you optimize a MERN application?

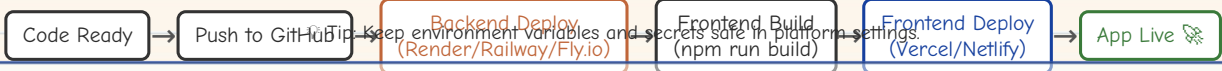
6. Production Best Practices

- Input Validation – frontend + backend.
- Error Handling – proper status codes and messages.
- Environment Variables – secrets in .env.
- HTTPS – always use in production.
- Rate Limiting – prevent abuse & brute force.
- Logging – morgan/winston for request logs.
- Secure JWT Storage – httpOnly cookies/localStorage.
- Password Hashing – bcrypt.

7. Common Folder Structure

```
client/src/ → components, pages, services, store, App.js
server/ → routes, controllers, models, middleware, config, .env, server.js
```

* 1. Deployment Guide (MERN App)



2. Useful NPM Packages

Package	Use
express	Web framework
mongoose	MongoDB ODM
cors	Enable CORS
dotenv	Load .env variables
jsonwebtoken	JWT sign & verify
bcryptjs	Hash passwords
axios	HTTP requests
nodemon	Auto restart server

3. MongoDB Aggregation Pipeline

```
db.orders.aggregate([
  { $match: { status: "completed" } },
  { $group: { _id: "$userId", total: { $sum: 1 } } },
  { $sort: { total: -1 } }
]);
// $match filter, $group aggregate, $sort order
```

4. Git Commands (Quick)

git init – Initialize repo
git add . – Stage changes
git commit -m "msg" – Commit
git branch – List branches
git checkout -b dev – Create & switch
git merge dev – Merge branch
git push / pull – Sync remote

5. React Performance Tips

- React.memo() to prevent re-renders.
- useCallback() & useMemo().
- Code-splitting with React.lazy().
- Optimize images and bundle size.

6. Node.js Best Practices

- ✓ Use async/await
- ✓ Keep routes and logic separate
- ✓ Handle errors with middleware
- ✓ Use helmet for security headers

7. Security Checklist

- ☐ Validate & sanitize user input
- ☐ Use HTTPS in production
- ☐ Store passwords hashed (bcrypt)
- ☐ Use environment variables
- ☐ Protect routes with JWT
- ☐ Limit requests (rate limiting)
- ☐ CORS configured properly
- ☐ Keep dependencies updated

mERN Stack Notes



3. Environment Variables (.env Example)

```
Client (.env):
VITE_API_URL=https://api.mydomain.com
VITE_APP_NAME=MyMERNApp

Server (.env):
PORT=5000
MONGO_URI=mongodb+srv://user:pass@cluster/mern
JWT_SECRET=mySuperSecretKey
CLIENT_URL=https://myfrontend.vercel.app
```

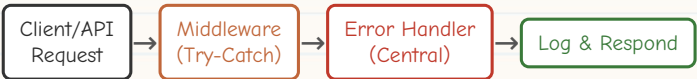
4. REST API Best Practices

- Use nouns, not verbs in endpoints.
- Keep URLs clean and consistent.
- Use proper status codes.
- Validate requests.
- Paginate and filter large data.
- Version your APIs (/api/v1/...).

5. Performance Optimization

- ✓ Code splitting in React (React.lazy)
- ✓ React.memo, useCallback, useMemo
- ✓ Optimize images (size, format, lazy load)
- ✓ Pagination, limit & projection in MongoDB
- ✓ Enable compression in Express
- ✓ Use caching (Redis / in-memory)

6. Error Handling Strategy



```
{ "success": false, "message": "Something went wrong" }
```

7. MongoDB Tips

- Create indexes on search/filter fields.
- Use .lean() for faster queries.
- Project only required fields.
- Regularly backup your database.

8. Git Workflow

git checkout -b feature/login

git commit -m "feat: login page"

git push origin feature/login

Create Pull Request

9. Useful Tools

- Postman / Thunder Client – API testing
- MongoDB Compass – DB GUI
- ESLint + Prettier – Code quality
- VS Code – Editor

★ 10. MERN Project Checklist: Authentication (Register/Login/Logout) ✓ Protected Routes ✓ CRUD Operations ✓ Form Validation ✓ Error Handling ✓